

VMI Photoelectron Analysis with Picasso

User Guide and Parameter Tuning Manual

VMI Analysis Pipeline Documentation

December 6, 2025

Abstract

This document provides comprehensive documentation for `process_andor_vmi_PICASSO.py`, a high-performance Velocity Map Imaging (VMI) photoelectron analysis pipeline based on the Picasso single-molecule localization microscopy (SMLM) software. The guide covers installation, configuration, parameter tuning, and interpretation of results.

Contents

1	Introduction	3
1.1	Overview	3
1.2	Key Features	3
1.3	Comparison with Other Methods	3
2	Installation	3
2.1	Prerequisites	3
2.2	Verify Picasso Installation	4
3	Configuration	4
3.1	Directory Structure	4
3.2	Configuration Parameters	4
4	Picasso Parameter Tuning	4
4.1	Critical Parameters	4
4.1.1	BOX_SIZE	4
4.1.2	MIN_NET_GRADIENT	5
4.2	Camera Parameters for MLE	6
4.2.1	CAMERA_BASELINE	6
4.2.2	CAMERA_SENSITIVITY	6
4.2.3	CAMERA_GAIN	6
4.2.4	CAMERA_QE	6
4.3	MLE Convergence Parameters	6
5	Running the Analysis	7
5.1	Tuning Mode (Recommended First Step)	7
5.2	Iterative Tuning Process	7
5.3	Batch Processing Mode	7

6	Understanding the Output	8
6.1	Output Files	8
6.2	Localization Data Columns	8
6.3	Interpreting Results	8
6.3.1	Total Event Count	8
6.3.2	Localization Precision	8
7	GMD Normalization	9
7.1	Overview	9
7.2	Configuration	9
7.3	Linear vs Nonlinear Processes	9
8	Advanced Options	9
8.1	Parallel Processing	9
8.2	Histogram Resolution	9
8.3	Abel Inversion	9
9	Troubleshooting	10
9.1	Common Issues	10
9.1.1	“CPU utilization was not set. Setting to 0.8”	10
9.1.2	Grid Artifacts in VMI Image	10
9.1.3	Spots Not Centered in Circles	10
9.1.4	RuntimeWarning: invalid value encountered in sqrt	10
9.1.5	Memory Error	10
9.2	Performance Optimization	10
10	Example Workflow	10
10.1	First-Time Analysis	10
11	References	11
12	Appendix: Parameter Quick Reference	12
13	Appendix: Picasso Algorithm Details	12
13.1	Net Gradient Algorithm	12
13.2	Maximum Likelihood Estimation	12

1 Introduction

1.1 Overview

The Picasso VMI analysis pipeline implements state-of-the-art centroiding and localization algorithms originally developed for super-resolution microscopy (DNA-PAINT, STORM, PALM). These methods are particularly well-suited for VMI photoelectron analysis due to:

- **Maximum Likelihood Estimation (MLE):** Statistically optimal fitting for Poisson-distributed photon counting data
- **Net Gradient Algorithm:** Superior detection of overlapping photoelectron events (pile-up recovery)
- **Numba JIT Compilation:** C-speed performance in pure Python
- **Sub-pixel Precision:** Typical precision of 0.01–0.1 pixels

1.2 Key Features

1. **Pile-up Recovery:** Net Gradient algorithm detects local maxima, enabling recovery of overlapping photoelectron events
2. **MLE Fitting:** Optimal for low-photon-count scenarios with Poisson noise
3. **GMD Normalization:** Shot-to-shot FEL pulse energy correction
4. **Dark Frame Correction:** Median dark frame subtraction
5. **Abel Inversion:** 3D momentum distribution reconstruction
6. **Parallel Processing:** Multi-core CPU utilization

1.3 Comparison with Other Methods

Method	Detection	Pile-up Recovery	Speed
Picasso	Net Gradient + MLE	Yes (local maxima)	Fast
PHOTUTILS	DAOStarFinder + Grouper	Yes (simultaneous fit)	Fast
SEP	Threshold + deblending	Limited	Fastest
Trackpy	Bandpass + centroid	No	Fast

Table 1: Comparison of VMI centroiding methods

2 Installation

2.1 Prerequisites

```
1 # Python 3.8 or higher
2 python --version
3
4 # Required packages
5 pip install picassosr numpy pandas tiff file matplotlib tqdm
6
7 # Optional packages
8 pip install pyabel # For Abel inversion
```

2.2 Verify Picasso Installation

```
1 python -c "from picasso.localize import identify_async; print('OK')"
```

3 Configuration

3.1 Directory Structure

```
1 VMI-FEL/TPX3/Andor/test/
2 |-- process_andor_vmi_PICASSO.py      # Main script
3 |-- ele0p812_60ev_5ms_20000/         # Data directory
4 |   |-- _1/Default/
5 |   |   |-- img_0000.tif
6 |   |   |-- img_0001.tif
7 |   |   |-- ...
8 |-- ele0p812_60ev_5ms_20000_B/       # Dark frames
9 |   |-- _1/Default/
10 |   |   |-- img_0000.tif
11 |   |   |-- ...
12 +-- vmi_analysis_picasso_ele0p812/   # Output (auto-created)
```

3.2 Configuration Parameters

Edit the configuration section in `process_andor_vmi_PICASSO.py`:

```
1 # --- INPUT/OUTPUT ---
2 DATA_DIR = "ele0p812_60ev_5ms_20000/_1/Default"
3 FILE_PATTERN = os.path.join(DATA_DIR, "img_*.tif")
4 OUTPUT_DIR = "vmi_analysis_picasso_ele0p812"
5
6 # --- DARK/BACKGROUND DATA ---
7 DARK_DATA_DIR = "ele0p812_60ev_5ms_20000_B/_1/Default"
8 DARK_FILE_PATTERN = os.path.join(DARK_DATA_DIR, "img_*.tif")
```

4 Picasso Parameter Tuning

4.1 Critical Parameters

4.1.1 BOX_SIZE

Definition: Size of the region of interest (ROI) for MLE fitting.

$$\text{BOX_SIZE} = \text{round}(6\sigma + 1) \quad (1)$$

where σ is the standard deviation of the photoelectron point spread function (PSF).

Typical Values:

- SMLM: 7–11 pixels ($\sigma \approx 1$ pixel)
- VMI: 7–13 pixels ($\sigma \approx 1 - 2$ pixels)

Guidelines:

- **Too small:** Truncates PSF wings, reduces precision

- **Too large:** Includes excess background noise, slows computation
- **Rule of thumb:** BOX_SIZE should contain 99% of photons

How to Tune:

1. Measure typical spot size in raw data (FWHM)
2. Calculate $\sigma = \text{FWHM}/2.355$
3. Set $\text{BOX_SIZE} = 6\sigma + 1$ (round to nearest odd integer)
4. Verify in tuning mode that localized spots are well-contained

4.1.2 MIN_NET_GRADIENT

Definition: Minimum threshold for Net Gradient detection metric.
The Net Gradient is calculated as:

$$\text{Net Gradient} = \sum_{i \in \text{inner}} I_i - \sum_{j \in \text{outer}} I_j \quad (2)$$

where the inner region is the central pixel, and the outer region is the surrounding annulus.

Properties:

- Roughly proportional to spot intensity
- Independent of local background
- Discriminates spots from noise

Typical Values:

- SMLM: 5000–10000
- VMI (high signal): 1000–3000
- VMI (low signal): 500–1000

Guidelines:

- **Too low:** False positives (noise detected as spots)
- **Too high:** Misses weak photoelectron events
- **Optimal:** Detects all real spots, zero background noise

How to Tune:

1. Run in tuning mode with initial guess (e.g., 1000)
2. Check detection visualization:
 - All bright spots should have red circles
 - Dark background should have no circles
3. Check Net Gradient histogram:
 - Should see clear separation between signal and noise
4. Iteratively adjust and re-run until satisfied

4.2 Camera Parameters for MLE

4.2.1 CAMERA_BASELINE

Definition: Average dark count level (ADU).

Important: Since dark frame subtraction is performed *before* Picasso localization, the baseline in the corrected image is effectively 0.

```
1 CAMERA_BASELINE = 0 # Already dark-subtracted
```

If dark subtraction is disabled: Set to the mean of your dark frame (e.g., 100–500 ADU).

4.2.2 CAMERA_SENSITIVITY

Definition: Conversion factor from ADU to electrons (e^- / ADU).

$$\text{Sensitivity} = \frac{\text{Electrons}}{\text{ADU}} \quad (3)$$

Typical Values:

- Andor iXon EMCCD: $\approx 6 e^-$ / ADU
- Andor Zyla sCMOS: $\approx 0.5 e^-$ / ADU
- Check camera specification sheet

For Andor SDK: Sensitivity is often returned in the metadata. If unavailable:

```
1 CAMERA_SENSITIVITY = 1.0 # Treat ADU as relative units
```

4.2.3 CAMERA_GAIN

Definition: EM gain (electron multiplication gain).

- **EMCCD with EM gain:** Set to actual gain (e.g., 200–300)
- **Conventional CCD/CMOS:** Set to 1

```
1 CAMERA_GAIN = 1 # No EM gain for conventional readout
```

4.2.4 CAMERA_QE

Definition: Quantum efficiency at detection wavelength.

Note: As of Picasso v0.6.0, this parameter is maintained for backward compatibility but not actively used in MLE fitting.

```
1 CAMERA_QE = 0.9 # Typical for back-illuminated CCDs
```

4.3 MLE Convergence Parameters

```
1 MLE_CONVERGENCE = 0.001 # Fitting tolerance
2 MLE_MAX_ITERATIONS = 1000 # Maximum iterations
```

MLE.CONVERGENCE: Smaller values = higher precision, longer computation. Default (0.001) is sufficient for most applications.

MLE.MAX.ITERATIONS: Spots that don't converge within this limit are still returned with their best-fit parameters.

5 Running the Analysis

5.1 Tuning Mode (Recommended First Step)

Before processing all frames, use tuning mode to optimize parameters on a single test frame.

```
1 # In process_andor_vmi_PICASSO.py
2 TUNING_ONLY = True
```

Run the script:

```
1 python process_andor_vmi_PICASSO.py
```

Output:

- 01_tuning_check_picasso.png: Visualization with 4 panels
 1. Raw frame with detected spots (red circles)
 2. Photon distribution histogram
 3. Net Gradient distribution histogram
 4. Localization precision histogram

What to Look For:

Panel	Quality Check
Raw Frame	All bright spots circled; no circles on dark background
Photons	Smooth distribution, no spike at threshold
Net Gradient	Clear separation between signal and noise; threshold at valley
Precision	Median < 0.1 pixels for good data

5.2 Iterative Tuning Process

1. Start with conservative guess: `MIN_NET_GRADIENT = 1000, BOX_SIZE = 7`
2. Run tuning mode
3. Evaluate detection quality:
 - **False positives (noise detected):** Increase `MIN_NET_GRADIENT`
 - **Missing weak spots:** Decrease `MIN_NET_GRADIENT`
 - **Spots truncated:** Increase `BOX_SIZE`
4. Adjust parameters and re-run
5. Repeat until detection is optimal

5.3 Batch Processing Mode

Once parameters are tuned, process all frames:

```
1 # In process_andor_vmi_PICASSO.py
2 TUNING_ONLY = False
```

Run the script:

```
1 python process_andor_vmi_PICASSO.py
```

Processing Time: Approximately 0.5–1 second per frame (depends on CPU, event density).

Example: 20,000 frames $\times$ 0.05 s/frame = 16 minutes on 4 cores.

6 Understanding the Output

6.1 Output Files

File	Description
vmi_results.h5	HDF5 file with all localization data (compressed)
vmi_results.csv	CSV file with all localization data (human-readable)
vmi_2d_raw.npy	2D VMI histogram (NumPy array)
vmi_3d_label.npy	Abel-inverted 3D distribution (if enabled)
04_final_summary.png	Summary visualization

6.2 Localization Data Columns

Column	Description
x, y	Sub-pixel coordinates (pixels)
mass	Integrated photon count
size	Gaussian PSF width (pixels)
ecc	Eccentricity: $1 - (\sigma_{\text{minor}}/\sigma_{\text{major}})$
signal	Peak intensity proxy
frame	Frame number
net_gradient	Net Gradient detection metric
gmd_energy	GMD pulse energy (if enabled)
weight	GMD normalization weight (if enabled)

6.3 Interpreting Results

6.3.1 Total Event Count

```
1 [Batch] Total detections: 5395255 events
2 [Batch] Events/frame: 269.8 avg
```

Typical Values:

- Low density: 10–100 events/frame
- Medium density: 100–500 events/frame
- High density: >500 events/frame (pile-up likely)

6.3.2 Localization Precision

Picasso reports localization precision from Cramér-Rao Lower Bound (CRLB):

$$\text{Precision} = \sqrt{\frac{\sigma^2}{N} + \frac{a^2}{12N} + \frac{8\pi\sigma^4b^2}{a^2N^2}} \quad (4)$$

where N = photon count, σ = PSF width, a = pixel size, b = background.

Typical VMI Values:

- High signal: 0.01–0.05 pixels
- Medium signal: 0.05–0.1 pixels
- Low signal: 0.1–0.3 pixels

7 GMD Normalization

7.1 Overview

FEL pulse energy fluctuates shot-to-shot. The Gas Monitor Detector (GMD) measures pulse energy for each shot. GMD normalization corrects for these fluctuations.

7.2 Configuration

```
1 ENABLE_GMD_WEIGHTING = True    # Enable normalization
2 ENABLE_GMD_WEIGHTING = False   # Disable (count all events equally)
```

7.3 Linear vs Nonlinear Processes

Linear Weighting (default):

$$w_i = \frac{\langle E_{\text{GMD}} \rangle}{E_{\text{GMD},i}} \quad (5)$$

Valid for single-photon ionization where signal $\propto$ intensity.

Nonlinear Processes (Strong-Field Ionization):

For N -photon ionization, signal $\propto I^N$. Linear weighting may be insufficient.

Recommendations:

- Pulse-to-pulse fluctuation $< 10\%$: Linear OK
- Pulse-to-pulse fluctuation $> 10\%$: Consider binning by GMD range
- Unknown nonlinearity: Set `ENABLE_GMD_WEIGHTING = False`, analyze bins separately

8 Advanced Options

8.1 Parallel Processing

```
1 ENABLE_PARALLEL = True
2 NUM_CORES = 4    # None = use all cores
```

Performance Scaling: Near-linear speedup (4 cores $\approx 4\times$ faster).

8.2 Histogram Resolution

```
1 HISTOGRAM_RESOLUTION_FACTOR = 1  # Native resolution (512 x 512)
2 HISTOGRAM_RESOLUTION_FACTOR = 2  # 2x upsampling (1024 x 1024)
```

Warning: Upsampling > 1 can create grid artifacts. Recommended: Keep at 1.

8.3 Abel Inversion

Enable 3D momentum distribution reconstruction:

```
1 pip install pyabel
```

```
1 # Will be automatically enabled if PyAbel is installed
```

9 Troubleshooting

9.1 Common Issues

9.1.1 “CPU utilization was not set. Setting to 0.8”

Cause: Picasso’s internal multiprocessing configuration.

Solution: This is now suppressed by setting `os.environ['PICASSO_CPU_UTILIZATION'] = '0.8'` before importing Picasso. If still appearing, ignore it (harmless).

9.1.2 Grid Artifacts in VMI Image

Cause: `HISTOGRAM_RESOLUTION_FACTOR > 1` causes interpolation artifacts.

Solution:

```
1 HISTOGRAM_RESOLUTION_FACTOR = 1 # Use native resolution
```

9.1.3 Spots Not Centered in Circles

Cause: Incorrect `CAMERA_BASELINE` when dark frame is already subtracted.

Solution:

```
1 CAMERA_BASELINE = 0 # For pre-subtracted dark frames
```

9.1.4 RuntimeWarning: invalid value encountered in sqrt

Cause: Negative CRLBs from unconverged fits.

Solution: Already suppressed with:

```
1 warnings.filterwarnings('ignore',  
2 message='invalid value encountered in sqrt')
```

Safe to ignore; unconverged fits are rare and don’t affect results.

9.1.5 Memory Error

Cause: Loading too many frames simultaneously.

Solution: The script uses on-demand loading. If still occurring, reduce `NUM_CORES`.

9.2 Performance Optimization

1. **Use SSD:** 5–10× faster than HDD for TIFF I/O
2. **Increase `NUM_CORES`:** Linear speedup up to physical core count
3. **Increase `MIN_NET_GRADIENT`:** Fewer detections = faster fitting
4. **Disable GMD weighting:** Saves 10–30 seconds for 20,000 frames

10 Example Workflow

10.1 First-Time Analysis

1. **Configure paths:**

```

1 DATA_DIR = "your_data/_1/Default"
2 DARK_DATA_DIR = "your_dark/_1/Default"
3 OUTPUT_DIR = "vmi_analysis_picasso_yourdata"

```

2. Enable tuning mode:

```

1 TUNING_ONLY = True
2 BOX_SIZE = 7
3 MIN_NET_GRADIENT = 1000

```

3. Run tuning:

```

1 python process_andor_vmi_PICASSO.py

```

4. Inspect 01_tuning_check_picasso.png

5. Adjust parameters and re-run until satisfied

6. Disable tuning mode:

```

1 TUNING_ONLY = False

```

7. Run full batch:

```

1 python process_andor_vmi_PICASSO.py

```

8. Inspect results:

- 04.final_summary.png: Visual summary
- vmi_results.csv: Localization data
- vmi_2d.raw.npy: VMI histogram

11 References

1. Schnitzbauer, J., Strauss, M. T., Schlichthaerle, T., Schueder, F., & Jungmann, R. (2017). Super-resolution microscopy with DNA-PAINT. *Nature Protocols*, 12(6), 1198–1228. <https://doi.org/10.1038/nprot.2017.024>
2. Sergé, A., Bertaux, N., Rigneault, H., & Marguet, D. (2008). Dynamic multiple-target tracing to probe spatiotemporal cartography of cell membranes. *Nature Methods*, 5(8), 687–694. <https://doi.org/10.1038/nmeth.1233>
3. Mortensen, K. I., Churchman, L. S., Spudich, J. A., & Flyvbjerg, H. (2010). Optimized localization analysis for single-molecule tracking and super-resolution microscopy. *Nature Methods*, 7(5), 377–381. <https://doi.org/10.1038/nmeth.1447>
4. Smith, C. S., Joseph, N., Rieger, B., & Lidke, K. A. (2010). Fast, single-molecule localization that achieves theoretically minimum uncertainty. *Nature Methods*, 7(5), 373–375. <https://doi.org/10.1038/nmeth.1449>
5. Picasso Documentation: <https://picassosr.readthedocs.io/>

12 Appendix: Parameter Quick Reference

Parameter	Description	Typical Value
BOX_SIZE	Fitting ROI size	7–13
MIN_NET_GRADIENT	Detection threshold	500–3000
CAMERA_BASELINE	Dark level (ADU)	0 (if dark-subtracted)
CAMERA_SENSITIVITY	e [−] / ADU conversion	1.0
CAMERA_GAIN	EM gain	1
CAMERA_QE	Quantum efficiency	0.9
MLE_CONVERGENCE	Fit tolerance	0.001
MLE_MAX_ITERATIONS	Max fit iterations	1000
NUM_CORES	CPU cores	4
ENABLE_GMD_WEIGHTING	GMD normalization	True/False
HISTOGRAM_RESOLUTION_FACTOR	Upsampling	1
TUNING_ONLY	Tuning mode	True/False

Table 2: Quick reference for critical parameters

13 Appendix: Picasso Algorithm Details

13.1 Net Gradient Algorithm

The Net Gradient is a local maximum detection algorithm:

1. Apply maximum filter with size = BOX_SIZE
2. Find pixels that are local maxima in their neighborhood
3. For each local maximum at (x_0, y_0) :

$$\text{NG} = I(x_0, y_0) \times |\nabla I| \quad (6)$$

4. Threshold by MIN_NET_GRADIENT

Advantages:

- Independent of local background
- Detects overlapping spots as separate local maxima
- Fast computation ($O(N \log N)$ with FFT convolution)

13.2 Maximum Likelihood Estimation

For each detected spot, fit 2D Gaussian PSF:

$$I(x, y) = A \exp \left(-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2} \right) + b \quad (7)$$

Maximize Poisson log-likelihood:

$$\mathcal{L} = \sum_{i,j} [d_{ij} \log \mu_{ij} - \mu_{ij} - \log(d_{ij}!)] \quad (8)$$

where d_{ij} = observed counts, μ_{ij} = model prediction.

Advantages of MLE:

- Achieves Cramér-Rao Lower Bound (theoretical optimum)
- Robust to low photon counts
- Provides uncertainty estimates (CRLBs)